

Introduction to Electronics Engineering

MODULE 4 — PART 1: COMPLETE NOTES

Module 4 (07 hrs): Introduction to Digital Electronics

For First-Year Students — written in simple, easy language
Course: Introduction to Electronics Engineering (IEE) · Course Code: EC 101

Module 4 — what it covers (as per the syllabus)

- Boolean Algebra
- Symbolic representation and Boolean algebraic functions
- Realization of basic logic gates using universal gates
- Adder and Subtractor

Contents

Section	Topic
1	Introduction to Digital Electronics and Boolean Algebra
2	Logic Gates (NOT, AND, OR, XOR, XNOR, NAND, NOR)
3	Properties and Laws of Boolean Algebra (with De Morgan's laws)
4	Universal Gates — realization using NAND and NOR
5	Circuits — Combinational and Sequential
6	Adders (half adder and full adder)
7	Half Subtractor
8	Standard Forms of Boolean Expressions (SOP and POS)
9	Complementing Boolean Functions
10	Algebraic Manipulation of Expressions
11	Quick Reference — Formulae and Definitions

1. Introduction to Digital Electronics and Boolean Algebra

1.1 What is Boolean Algebra?

Boolean algebra is a branch of mathematics dealing with variables that have only two values: **True (1)** or **False (0)**. It was introduced by George Boole in 1854 and forms the mathematical foundation of all digital circuits.

There are **three basic operations** in Boolean algebra:

- **AND** — written as $x \cdot y$ (or simply xy)
- **OR** — written as $x + y$
- **NOT** — written as x' (complement)

- Introduced by George Boole in 1854
- Three basic operations: **AND**, **OR**, and **NOT**
- The **AND** operator is denoted by a dot ($\cdot$),
 $x \cdot y$ or xy is read: x **AND** y
- The **OR** operator is denoted by a plus ($+$)
 $x + y$ is read: x **OR** y
- The **NOT** operator is denoted by ($'$) or an overbar ($\bar{}$).
 x' or $\bar{x}$ is the complement of x

$x \cdot y$ is the **AND** operator

$x + y$ is the **OR** operator

x' is the **NOT** operator

Figure 1: Boolean algebra — the three basic operations and their symbols.

AND ($\cdot$)	OR ($+$)	NOT ($\bar{}$)
$0 \cdot 0 = 0$	$0 + 0 = 0$	$\bar{0} = 1$
$0 \cdot 1 = 0$	$0 + 1 = 1$	$\bar{1} = 0$
$1 \cdot 0 = 0$	$1 + 0 = 1$	
$1 \cdot 1 = 1$	$1 + 1 = 1$	
Multiplication and AND give identical results	Addition and OR give different Results for 1+1	No corresponding Math operator for NOT

Figure 2: Truth tables of the AND, OR and NOT operations.

1.2 Importance of Boolean Algebra

- A **logic gate** is a device that performs a Boolean function — a logical operation performed on one or more binary inputs that produces a single binary output.
- Digital circuits use signals with two possible values: **logic 0** is a low voltage signal (around 0 volts) and **logic 1** is a high voltage signal (e.g. 5 V or 3.3 V).
- The **physical value** of a signal is the actual voltage it carries, while its **logic value** is either 0 (low) or 1 (high).
- Having only two logic values (0 and 1) simplifies the implementation of digital circuits.

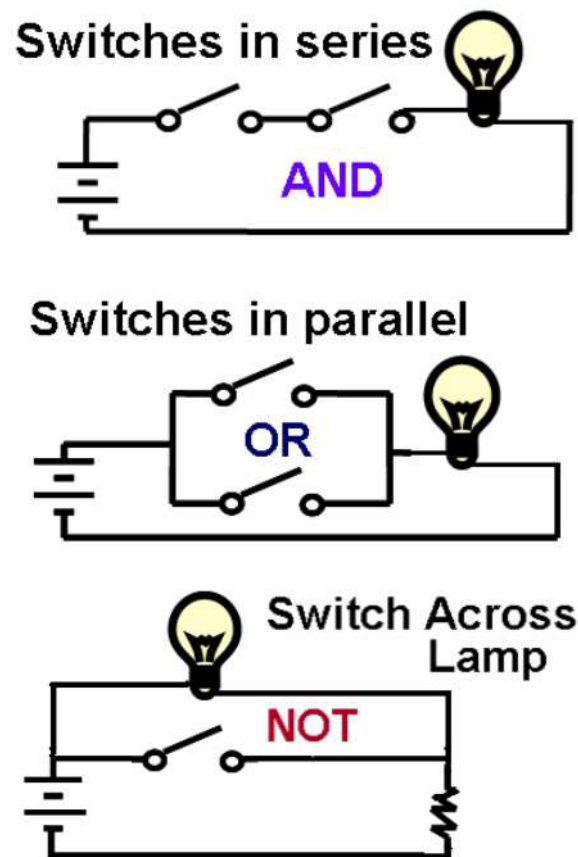
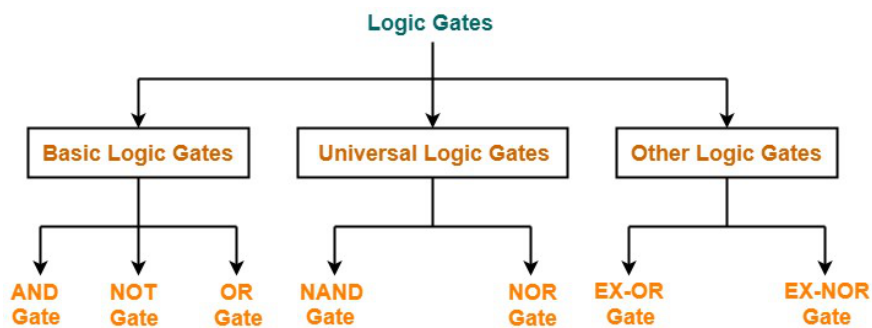


Figure 3: AND, OR and NOT functions represented by simple switches.

2. Logic Gates

2.1 What is a Logic Gate?

- A gate is a device that performs a basic operation on electrical signals.
- Gates are combined into circuits to perform more complicated tasks.
- The behaviour of gates and circuits can be described in three ways: **Boolean expressions**, **logic diagrams**, and **truth tables**.
- There are **seven types** of gates: NOT, AND, OR, XOR, XNOR, NAND and NOR.



Types of Logic Gates

Figure 4: Classification of the logic gates.

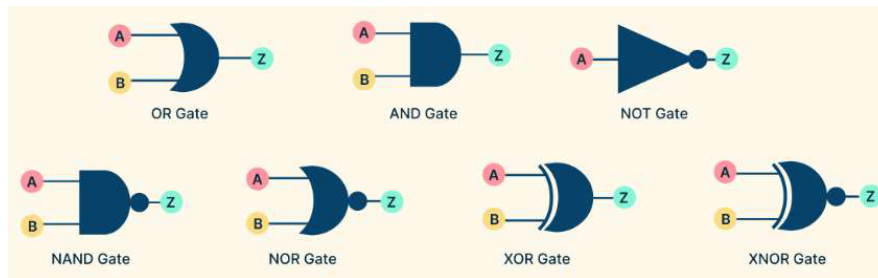


Figure 5: Symbols of the basic logic gates.

2.2 The NOT Gate

- A NOT gate accepts one input value and produces one output value.
- If the input is 0, the output is 1; if the input is 1, the output is 0.
- It is also called an **inverter** because it inverts the input value: $Y = A'$.

Boolean Expression	Logic Diagram Symbol	Truth Table						
$X = A'$		<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0
A	X							
0	1							
1	0							

Figure 6: NOT gate — symbol, Boolean expression and truth table.

2.3 The AND Gate

An AND gate accepts two input signals. If both inputs are 1, the output is 1; otherwise the output is 0. Boolean expression: $Y = A \cdot B$.

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A \cdot B$		<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>X</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>0</td></tr> <tr> <td>0</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>0</td></tr> <tr> <td>1</td><td>1</td><td>1</td></tr> </tbody> </table>	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1
A	B	X															
0	0	0															
0	1	0															
1	0	0															
1	1	1															

Figure 7: AND gate — symbol, Boolean expression and truth table.

2.4 The OR Gate

If both inputs are 0, the output is 0; otherwise the output is 1. Boolean expression: $Y = A + B$.

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A + B$		<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>X</th></tr> </thead> <tbody> <tr> <td>0</td><td>0</td><td>0</td></tr> <tr> <td>0</td><td>1</td><td>1</td></tr> <tr> <td>1</td><td>0</td><td>1</td></tr> <tr> <td>1</td><td>1</td><td>1</td></tr> </tbody> </table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	1															

Figure 8: OR gate — symbol, Boolean expression and truth table.

2.5 The XOR Gate

- XOR stands for **exclusive OR**.
- An XOR gate produces 0 if its two inputs are the same, and 1 otherwise.
- Note the difference between XOR and OR: they differ in only one input situation — when both inputs are 1, the OR gate produces 1 but the XOR produces 0.
- Boolean expression: $Y = A \oplus B = A'B + AB'$.

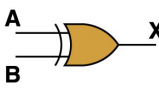
Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A \oplus B$		<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>X</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	0															

Figure 9: XOR gate — symbol, Boolean expression and truth table.

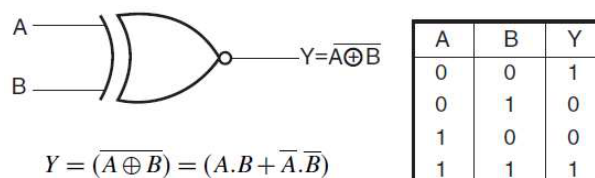


Figure 10: XOR gate symbol with its Boolean expression.

2.6 The XNOR Gate

The XNOR gate is the complement of the XOR gate. It produces 1 when both inputs are the same, and 0 otherwise. Boolean expression: $Y = (A \oplus B)'$.

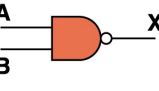
Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = (A \cdot B)'$		<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>X</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	1															
0	1	1															
1	0	1															
1	1	0															

Figure 11: XNOR gate — symbol, Boolean expression and truth table.

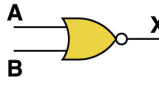
Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = (A + B)'$		<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>X</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X															
0	0	1															
0	1	0															
1	0	0															
1	1	0															

Figure 12: XNOR gate — symbol and truth table.

2.7 The NAND and NOR Gates

The NAND and NOR gates are essentially the **opposite** of the AND and OR gates respectively. NAND: $Y = (A \cdot B)'$; NOR: $Y = (A + B)'$.

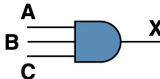
Boolean Expression	Logic Diagram Symbol	Truth Table																
$X = A \cdot B \cdot C$		<table><tr><th>A</th><th>B</th><th>C</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	A	B	C	X	0	0	0	0	0	0	1	0	0	1	0	0
A	B	C	X															
0	0	0	0															
0	0	1	0															
0	1	0	0															

Figure 13: NAND and NOR gate symbols with their truth tables.

A	B	C	X
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Figure 14: NAND and NOR gate truth tables.

2.8 Review of Gate Processing

- A NOT gate inverts its single input value.
- An AND gate produces 1 if both input values are 1.
- An OR gate produces 1 if one or the other or both input values are 1.
- An XOR gate produces 1 if one or the other (but not both) input values are 1.
- A NAND gate produces the opposite result of an AND gate.
- A NOR gate produces the opposite result of an OR gate.

2.9 Gates with More Inputs

Gates can be designed to accept three or more input values. A three-input AND gate, for example, produces an output of 1 only if **all** input values are 1.

3. Properties and Laws of Boolean Algebra

3.1 The Laws of Boolean Algebra

1. Closure: the result of any Boolean operation is in $B = \{0, 1\}$
2. Identity element with respect to $+$ is 0: $x + 0 = 0 + x = x$
Identity element with respect to $\cdot$ is 1: $x \cdot 1 = 1 \cdot x = x$
3. Commutative with respect to $+$: $x + y = y + x$
Commutative with respect to $\cdot$: $x \cdot y = y \cdot x$
4. $\cdot$ is distributive over $+$: $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$

Figure 15: Basic properties of Boolean algebra.

$+$ is distributive over $\cdot$: $x + (y \cdot z) = (x + y) \cdot (x + z)$

5. For every x in B , there exists x' in B (called complement of x) such that: $x + x' = 1$ and $x \cdot x' = 0$

Figure 16: Further Boolean-algebra properties.

Property	Form 1 (OR / AND)	Dual Form
Identity	$x + 0 = x$	$x \cdot 1 = x$
Complement	$x + x' = 1$	$x \cdot x' = 0$
Null	$x + 1 = 1$	$x \cdot 0 = 0$
Idempotence	$x + x = x$	$x \cdot x = x$
Involution	$(x')' = x$	—
Commutative	$x + y = y + x$	$x \cdot y = y \cdot x$
Associative	$(x + y) + z = x + (y + z)$	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$
Distributive	$x \cdot (y + z) = x \cdot y + x \cdot z$	$x + y \cdot z = (x + y)(x + z)$
Absorption	$x + x \cdot y = x$	$x \cdot (x + y) = x$
Simplification	$x + x' \cdot y = x + y$	$x \cdot (x' + y) = x \cdot y$
De Morgan	$(x + y)' = x' \cdot y'$	$(x \cdot y)' = x' + y'$

3.2 De Morgan's Laws

De Morgan's theorems allow us to convert an AND of complemented terms into an OR of uncomplemented terms, and vice versa:

$$(A + B)' = A' \cdot B' \quad (A \cdot B)' = A' + B'$$

- **First law:** the complement of a sum equals the product of the complements.
- **Second law:** the complement of a product equals the sum of the complements.

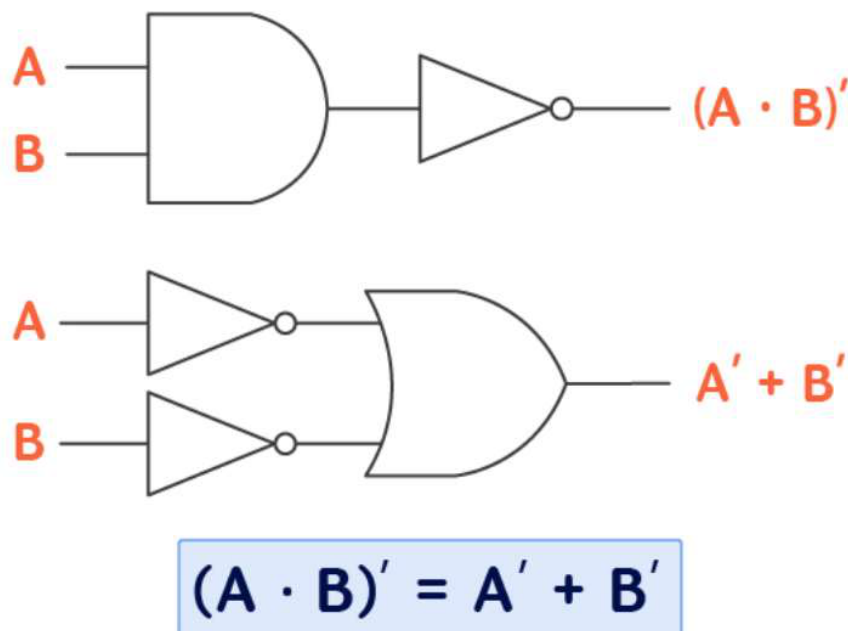


Figure 17: De Morgan's laws realised with logic gates.

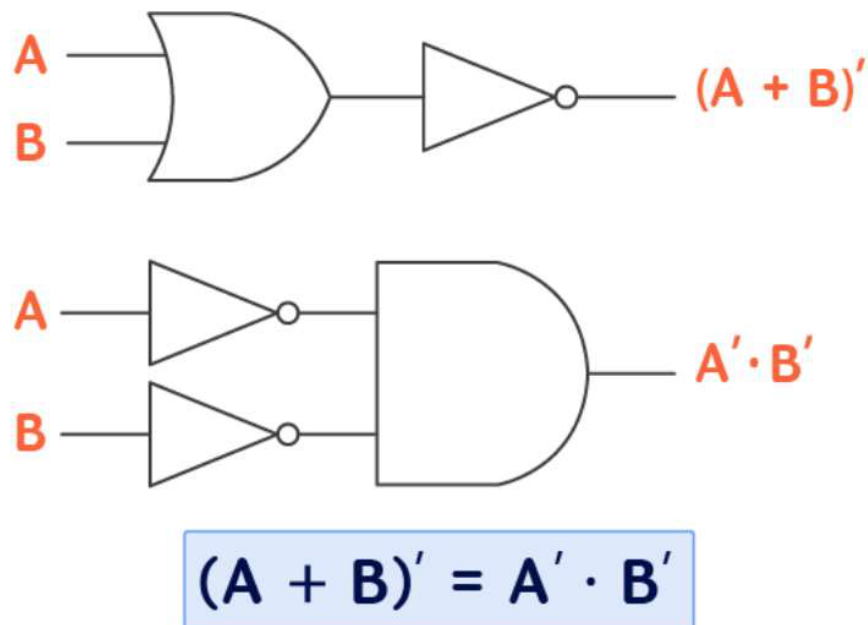


Figure 18: Truth table verifying the first De Morgan law.

Inputs			Outputs			
A	B	$A \cdot B$	$(A \cdot B)'$	A'	B'	$A' + B'$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Figure 19: Truth table verifying the second De Morgan law.

4. Universal Gates — Realization Using NAND and NOR

The NAND and NOR gates are **self-sufficient** — any logic circuit can be built using only NAND gates, or only NOR gates. Therefore NAND and NOR are called **universal gates** (a complete set of logic). They can be used to replace an AND gate, an OR gate, or an inverter.

4.1 Realization Using NAND Gates

(a) NOT gate using NAND

Tie both inputs of the NAND gate together: $Z = (X \cdot X)' = X'$. The circuit is equivalent to an inverter.

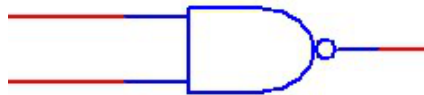


Figure 20: NOT gate realized using a NAND gate.

(b) AND gate using NAND

A NAND gate followed by an inverter (a second NAND with inputs tied) gives the AND function: $Z = ((X \cdot Y)')' = X \cdot Y$.

- Implementing AND using NAND gates:

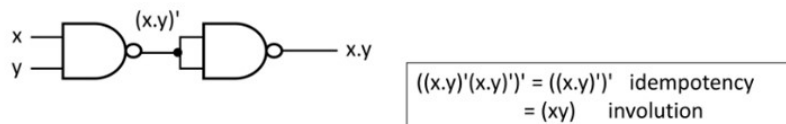


Figure 21: AND gate realized using two NAND gates.

(c) OR gate using NAND

Invert both inputs, then apply NAND: $Z = X' \cdot Y' = (X + Y)'$. The circuit is equivalent to an OR gate.

- Implementing OR using NAND gates:

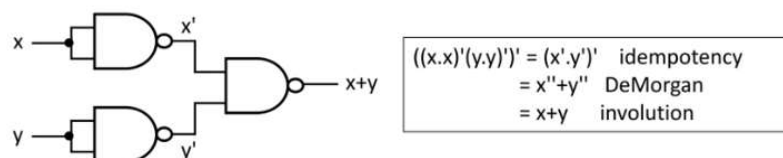


Figure 22: OR gate realized using three NAND gates.

(d) NAND gate equivalent to the AOI gates

Using only NAND gates we can build the AND, OR and inverter (AOI) functions — this is why NAND is a universal gate.

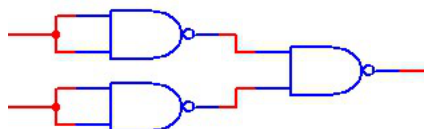


Figure 23: Realization of the basic gates using NAND gates.

4.2 Realization Using NOR Gates

(a) NOT gate using NOR

Tie both inputs of the NOR gate together: $Z = (X + X)' = X'$.

- NOR Gate as an Inverter



Figure 24: NOT gate realized using a NOR gate.

(b) OR gate using NOR

A NOR gate followed by an inverter gives the OR function: $Z = ((X + Y)')' = X + Y$.

- Implementing OR using NOR gates:

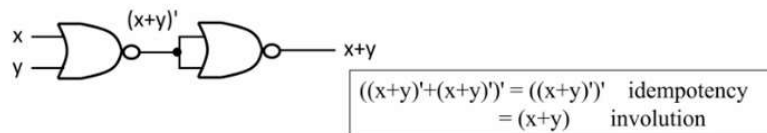


Figure 25: OR gate realized using NOR gates.

(c) AND gate using NOR

Invert both inputs, then apply NOR: $Z = X' + Y' = (X \cdot Y)'$. The circuit is equivalent to an AND gate.

- Implementing AND using NOR gates:

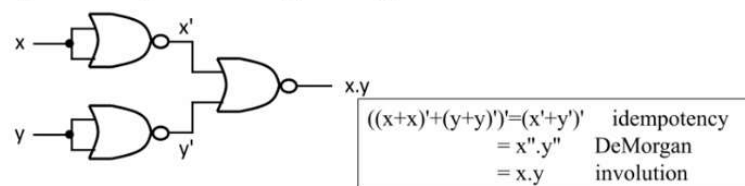


Figure 26: AND gate realized using NOR gates.

5. Circuits — Combinational and Sequential

- Digital circuits fall into two general categories.
- In a **combinational circuit**, the input values explicitly determine the output.
- In a **sequential circuit**, the output is a function of the input values **as well as** the existing state of the circuit.
- As with gates, the operation of entire circuits can be described using three notations: Boolean expressions, logic diagrams, and truth tables.

5.1 Combinational Circuit Example

Gates are combined into circuits by using the output of one gate as the input for another. Because there are three inputs to this circuit, eight rows are required to describe all possible input combinations. The same circuit in Boolean algebra is $(AB + AC)$.

A	B	C	D	E	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Figure 27: A combinational circuit and its truth table for the function $AB + AC$.

6. Adders

6.1 Half Adder

- At the digital logic level, addition is performed in binary, using special circuits called **adders**.
- The result of adding two binary digits can produce a carry value (recall that $1 + 1 = 10$ in base two).
- A circuit that computes the sum of two bits and produces the correct carry bit is called a **half adder**.



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 28: Half adder — block diagram and truth table.

Two Boolean expressions:

$$\text{Sum} = A \oplus B \quad \text{Carry} = A \cdot B$$

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

Logic Circuit

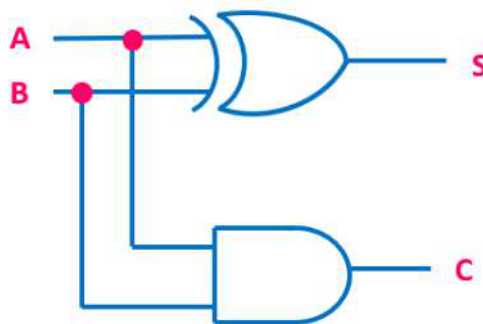


Figure 29: Half adder logic circuit using an XOR gate and an AND gate.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 30: Half adder truth table.

6.2 Full Adder

A **full adder** is a combinational circuit that forms the arithmetic sum of three input bits; it consists of **three inputs** (A, B, C_{in}) and **two outputs** (Sum and Carry-out).



Figure 31: Block diagram of a full adder.

A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 32: Full adder truth table.

Boolean expressions:

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}$$

$$\text{Carry-out} = A \cdot B + C_{\text{in}}(A \oplus B)$$

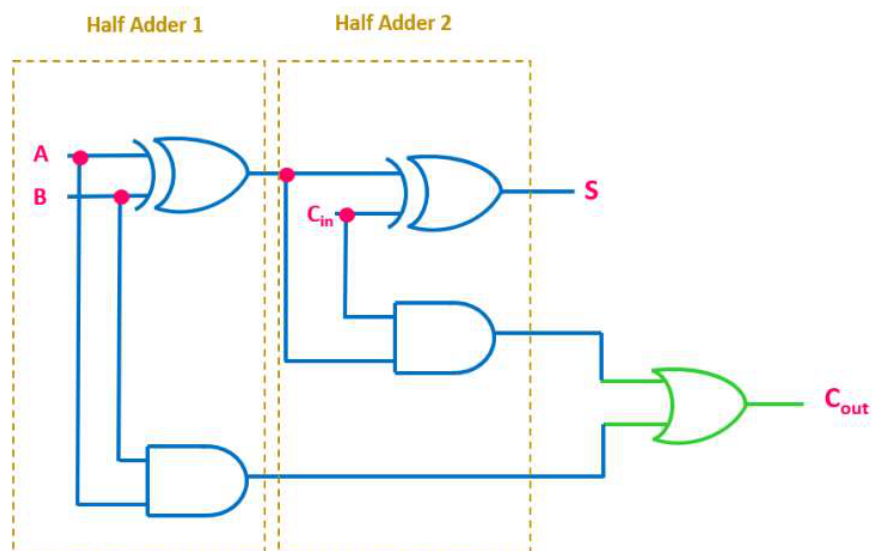


Figure 33: Full adder built from two half adders and an OR gate.

7. Half Subtractor

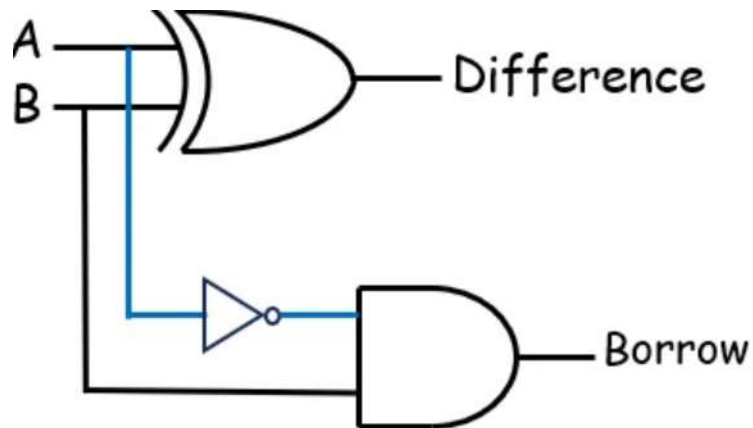
A **half subtractor** is a digital circuit designed to perform the subtraction of two single-bit binary numbers. It produces two outputs: a **difference bit (D)** and a **borrow bit (B)**.

Logical expressions of the outputs:

$$\text{Difference } D = A \oplus B \quad \text{Borrow } B = A' \cdot B$$

Components of a half subtractor:

- **XOR gate** — computes the difference bit (D).
- **AND gate** (with an inverted A) — computes the borrow bit (B).



Half Subtractor Circuit Diagram

Figure 34: Half subtractor circuit diagram.

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Figure 35: Half subtractor truth table.

8. Standard Forms of Boolean Expressions

Usually, literals (Boolean variables in complemented or uncomplemented form) and terms are arranged in one of the two standard forms of switching equations:

- **Sum of Products (SOP)** — a set of product (AND) terms that are summed (OR) together. It uses minterms.
- **Product of Sums (POS)** — a set of sum (OR) terms that are multiplied (AND) together. It uses maxterms.

8.1 Standard vs. Canonical Forms

Feature	Standard Form	Canonical Form
SOP	May have missing literals in product terms	Every product term contains ALL variables
POS	May have missing literals in sum terms	Every sum term contains ALL variables
Example	$F = AB + AC$ (Standard SOP)	$F = A'BC + AB'C + ABC' + ABC$ (Canonical SOP)

In Digital Electronics, the canonical form means writing a Boolean function so that each term contains all variables exactly once (either complemented or uncomplemented).

- **Minterms** give the Canonical SOP form.
- **Maxterms** give the Canonical POS form.

Variables			Minterms	Maxterms
A	B	C	m_i	M_i
0	0	0	$\bar{A} \bar{B} \bar{C} = m_0$	$A + B + C = M_0$
0	0	1	$\bar{A} \bar{B} C = m_1$	$A + B + \bar{C} = M_1$
0	1	0	$\bar{A} B \bar{C} = m_2$	$A + \bar{B} + C = M_2$
0	1	1	$\bar{A} B C = m_3$	$A + \bar{B} + \bar{C} = M_3$
1	0	0	$A \bar{B} \bar{C} = m_4$	$\bar{A} + B + C = M_4$
1	0	1	$A \bar{B} C = m_5$	$\bar{A} + B + \bar{C} = M_5$
1	1	0	$A B \bar{C} = m_6$	$\bar{A} + \bar{B} + C = M_6$
1	1	1	$A B C = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$

Table 4.7 Minterms and Maxterms for three variables

Figure 36: Minterms and maxterms for three variables.

8.2 SOP vs. POS

S.No.	Sum of Products (SOP)	Product of Sums (POS)
1	SOP stands for Sum of Products.	POS stands for Product of Sums.
2	Defines Boolean terms as the sum of product terms.	Defines Boolean terms as a product of sum terms.
3	It prefers minterms.	It prefers maxterms.
4	Minterms are denoted by 'm'.	Maxterms are denoted by 'M'.
5	It gives a HIGH (1) output.	It gives a LOW (0) output.
6	The final term is obtained by adding the product terms.	The final term is obtained by multiplying the sum terms.

8.3 Worked Examples — Conversion to Canonical Form

Example 1: Convert $Y = AC + AB + BC$ into canonical SOP form

Multiply each term by the missing variable in the form $(X + X') = 1$:

$$\begin{aligned}
 Y &= AC(B + B') + AB(C + C') + BC(A + A') \\
 &= ABC + AB'C + ABC + ABC' + ABC + A'BC \\
 &= A'BC + AB'C + ABC' + ABC \quad [\text{using } A + A = A]
 \end{aligned}$$

$$Y = AC + AB + BC$$

$$\begin{aligned}
 &= AC (B + \bar{B}) + AB (C + \bar{C}) + BC (A + \bar{A}) \\
 &\quad [\because A + \bar{A} = 1; B + \bar{B} = 1; C + \bar{C} = 1] \\
 &= ABC + A\bar{B}\bar{C} + ABC + AB\bar{C} + ABC + \bar{A}BC \\
 &= A\bar{B}\bar{C} + AB\bar{C} + ABC + \bar{A}BC \quad \text{Rule : } [A + A = A]
 \end{aligned}$$

Figure 37: Canonical SOP conversion — Example 1.

Example 2: Convert $Y = A + AB + ABC$ into canonical SOP form

$$\begin{aligned}
 Y &= A(B + B')(C + C') + AB(C + C') + ABC \\
 &= ABC + AB'C + ABC' + AB'C' \quad [\text{using } A + A = A]
 \end{aligned}$$

$$\begin{aligned}
 Y &= A + AB + ABC \\
 &= A \cdot (B + \bar{B}) \cdot (C + \bar{C}) + AB \cdot (C + \bar{C}) + ABC \\
 &= (AB + A\bar{B}) \cdot (C + \bar{C}) + ABC + AB\bar{C} + ABC \\
 &= ABC + AB\bar{C} + A\bar{B}\bar{C} + A\bar{B}C + ABC + AB\bar{C} + ABC \\
 &= ABC + AB\bar{C} + A\bar{B}\bar{C} + A\bar{B}C \quad \text{Rule : } [A + A = A]
 \end{aligned}$$

Figure 38: Canonical SOP conversion — Example 2.

Example 3: Convert $Y = (A + B)(B + C)(A + C)$ into canonical POS form

Add the missing variable in the form $(X \cdot X') = 0$ to each sum term:

$$\begin{aligned} Y &= (A + B + C \cdot C')(B + C + A \cdot A')(A + C + B \cdot B') \\ &= (A + B + C)(A + B + C')(A' + B + C)(A + B' + C) \end{aligned}$$

$$\begin{aligned} Y &= (A + B) (B + C) (A + C) \\ &= (A + B + C \cdot \bar{C}) (B + C + A \cdot \bar{A}) (A + C + B \cdot \bar{B}) \\ &= (A + B + C)(A + B + \bar{C})(A + B + C)(\bar{A} + B + C)(A + B + C)(A + \bar{B} + C) \\ &= (A + B + C) (A + B + \bar{C}) (\bar{A} + B + C) (A + \bar{B} + C) \end{aligned}$$

Figure 39: Canonical POS conversion — Example 3.

Example 4: Convert $Y = A(A + B)(A + B + C)$ into canonical POS form

$$\begin{aligned} Y &= (A + B \cdot B' + C \cdot C')(A + B + C \cdot C')(A + B + C) \\ &= (A + B + C)(A + B' + C)(A + B + C')(A + B' + C') \end{aligned}$$

$$\begin{aligned} Y &= A (A + B) (A + B + C) \\ &= (A + B \cdot \bar{B} + C \cdot \bar{C}) (A + B + C \cdot \bar{C}) (A + B + C) \\ &\quad [\because B \cdot \bar{B} = 0 ; C \cdot \bar{C} = 0] \\ &= (A + B \cdot \bar{B} + C) (A + B \cdot \bar{B} + C) (A + B + C) (A + B + \bar{C}) (A + B + C) \\ &= (A + B + C) (A + \bar{B} + C) (A + B + \bar{C}) (A + \bar{B} + \bar{C}) \\ &\quad (A + B + C) (A + B + \bar{C}) (A + B + C) \\ &= (A + B + C) (A + \bar{B} + C) (A + B + \bar{C}) (A + \bar{B} + \bar{C}) \end{aligned}$$

Figure 40: Canonical POS conversion — Example 4.

9. Complementing Boolean Functions

To complement a Boolean function, we **interchange the AND and OR operators** and complement every literal. De Morgan's theorems are used for this.

Example 1: what is the complement of $f = x'yz' + xy'z'$?

$$f' = (x + y' + z)(x' + y + z)$$

Example 2: complement $g = (a' + bc)d' + e$

$$g' = (a(b' + c') + d) e'$$

- What is the complement of $f = x'yz' + xy'z'$?
- Use DeMorgan's Theorem:
 - Complement each variable and constant

Figure 41: Complementing a Boolean function — statement.

- Interchange AND and OR operators
- So, what is the complement of $f = x'yz' + xy'z'$?

Answer: $f' = (x + y' + z)(x' + y + z)$

- **Example 2:** Complement $g = (a' + bc)d' + e$
- **Answer:** $g' = (a(b' + c') + d)e'$

Figure 42: Complementing Boolean functions — worked answers.

10. Algebraic Manipulation of Expressions

Boolean expressions can be simplified (manipulated) using the laws of Boolean algebra, which reduces the number of gates needed to implement a circuit. The worked example below shows the step-by-step simplification of a Boolean expression.

- The objective is to acquire skills in manipulating Boolean expressions, to transform them into simpler form.
- **Example 1:** prove $x + xy = x$ (absorption theorem)
- **Proof:**

$$\begin{aligned}
 x + xy &= x \cdot 1 + xy & x \cdot 1 &= x \\
 &= x \cdot (1 + y) & \text{Distributive } \cdot \text{ over } + \\
 &= x \cdot 1 = x & (1 + y) &= 1
 \end{aligned}$$
- **Example 2:** prove $x + x'y = x + y$ (simplification theorem)
- **Proof:**

$$\begin{aligned}
 x + x'y &= (x + x')(x + y) & \text{Distributive } + \text{ over } \cdot \\
 &= 1 \cdot (x + y) & (x + x') &= 1 \\
 &= x + y
 \end{aligned}$$

Figure 43: Algebraic manipulation of a Boolean expression — worked example.

11. Quick Reference — Key Formulae and Definitions

11.1 Key Gate Expressions

Gate	Boolean Expression	Output is 1 when...
NOT	$Y = A'$	Input is 0
AND	$Y = A \cdot B$	Both inputs are 1
OR	$Y = A + B$	At least one input is 1
NAND	$Y = (A \cdot B)'$	Not both inputs are 1
NOR	$Y = (A + B)'$	Both inputs are 0
XOR	$Y = A \oplus B = A'B + AB'$	Inputs are different
XNOR	$Y = (A \oplus B)' = AB + A'B'$	Inputs are the same

11.2 Key Formulae

Quantity	Formula	Notes
Half adder	Sum = $A \oplus B$; Carry = $A \cdot B$	Two inputs, two outputs
Full adder	Sum = $A \oplus B \oplus C_{in}$; Carry = $AB + C_{in}(A \oplus B)$	Three inputs, two outputs
Half subtractor	Difference $D = A \oplus B$; Borrow $B = A' \cdot B$	Two inputs, two outputs
De Morgan (1st)	$(A + B)' = A' \cdot B'$	Complement of a sum
De Morgan (2nd)	$(A \cdot B)' = A' + B'$	Complement of a product
Universal gates	NAND and NOR	Each can build any logic circuit

11.3 Key Definitions at a Glance

Term	Definition
Boolean algebra	A branch of mathematics dealing with variables that have only two values, True (1) or False (0).
Logic gate	A device that performs a basic Boolean operation on one or more binary inputs to give a single binary output.
Truth table	A table listing the output of a logic circuit for every possible combination of inputs.
Universal gate	A gate (NAND or NOR) using which any logic circuit can be built.
Combinational circuit	A circuit in which the output depends only on the present inputs.
Sequential circuit	A circuit in which the output depends on the inputs and the existing state of the circuit.
SOP	Sum of Products — a set of product terms summed together (uses minterms).
POS	Product of Sums — a set of sum terms multiplied together (uses maxterms).
Canonical form	A standard form in which every term contains all the variables exactly once.
Minterm	A product term containing all variables exactly once (canonical SOP).
Maxterm	A sum term containing all variables exactly once (canonical POS).
Half adder	A circuit that adds two single bits and produces a sum and a carry.
Full adder	A circuit that adds three bits (A , B , C_{in}) and produces a sum and a carry-out.
Half subtractor	A circuit that subtracts two single bits and produces a difference and a borrow.